

Praktikum 5: B-Baum

Wir haben in der Vorlesung den B-Baum als stets balancierten Suchbaum kennengelernt, dessen Knoten mehr als einen Schlüssel enthalten können. Im Gegensatz zum Binären Suchbaum sind B-Bäume für den Einsatz auf externen Speichermedien (Festplatten, SSDs) optimiert: ein großes Verzweigungsverhältnis hält die Baumhöhe gering und minimiert damit die Zahl der Plattenzugriffe.

Die vollständige Implementierung (`BTree`, `BTreeNode`) finden Sie in `vorlesung/L06_b_baeume/`. Alle Aufgaben bauen darauf auf — Sie implementieren oder analysieren, ohne den Vorlesungscode zu verändern.

Aufgabe 1: Graphviz-Visualisierung

Implementieren Sie eine Unterklasse `BTreeVis(BTree)`, die eine Methode `graph_traversal()` besitzt. Diese soll den B-Baum mit Hilfe von Graphviz als PDF ausgeben. Stellen Sie jeden Knoten als Auflistung der enthaltenen Werte dar; die Kanten verbinden den Knoten mit den entsprechenden Kindknoten, müssen aber nicht von einer bestimmten Position ausgehen.

Überprüfen Sie Ihre Implementierung, indem Sie die Sequenz `seq1.txt` in einen B-Baum der Ordnung 3 einfügen und die Visualisierung erzeugen.

- Wie viele Schlüssel enthält die Wurzel?
- Liegen alle Blattknoten auf derselben Tiefe? Warum ist das bei einem B-Baum immer so?

Aufgabe 2: Eigenschaften des B-Baums

Lesen Sie `seq3.txt` ein und fügen Sie die Werte jeweils in einen B-Baum der Ordnung 3 und der Ordnung 5 ein.

- Welche Höhe ergibt sich jeweils?
- Traversieren Sie beide Bäume und überprüfen Sie, ob die Schlüssel in sortierter Reihenfolge ausgegeben werden.
- Suchen Sie jeweils den Knoten, der den Wert 0 enthält, und geben Sie seinen kompletten Inhalt aus. Warum enthält der Knoten bei Ordnung 5 tendenziell mehr Schlüssel als bei Ordnung 3?

Aufgabe 3: Disk-I/O und der Parameter m

Die Klasse `BTreeNode` zählt in den Attributen `loaded_count` und `saved_count`, wie oft ein Knoten von der Festplatte geladen bzw. geschrieben wurde. Analysieren Sie, wie diese Zählwerte sowie die Anzahl der Vergleiche und die Baumhöhe von der Ordnung m abhängen.

Fügen Sie dazu eine zufällig generierte Sequenz von 500 Werten in B-Bäume der Ordnungen 2, 3, 5, 10 und 20 ein und stellen Sie die Ergebnisse in einem Plot dar.

- Warum sinkt die Zahl der Plattenzugriffe mit wachsendem m ?
- Warum steigt die Zahl der internen Vergleiche mit wachsendem m , obwohl die Baumhöhe sinkt? Leiten Sie die asymptotische Anzahl der Vergleiche pro Einfügung in Abhängigkeit von m her.
- Festplatten lesen typischerweise Blöcke von 4 KB. Ein Knoten soll genau in einen Block passen. Die Schlüssel sind 8-Byte-Integer-Werte; die Kindzeiger sind Blocknummern auf dem Speichermedium (ebenfalls 8 Byte). Welche Ordnung m ist optimal, und wie lässt sich das herleiten?